

— Databases

Hoofdstuk 1: Inleiding

L. Espeel



— Doelstelling

Praktisch alle computerprogramma's gebruiken **data**, in deze cursus bekijken we voornamelijk onderstaande:

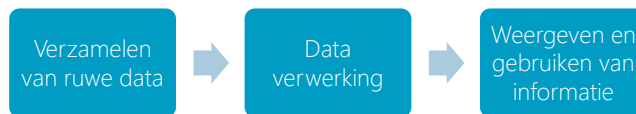
- Structuur van een relationele databank
- Relaties tussen verschillende elementen in een databank
- Data opvragen met SQL-commando's en data filteren, sorteren, groeperen, ...
- Interactief omgaan met data in een databank met behulp van een DBMS-tool
- ...

Cursusmateriaal is samenstelling uit cursussen van L. Espeel, M. Leroy en T. Termote.

— Data (1)

Wat?

- **Ruwe data** (gegevens): zijn feiten en cijfers, al dan niet gestructureerd.
 - Echter weinig betekenis totdat ze verwerkt zijn.
- **Dataverwerking**: bv. filteren en sorteren van data.
- **Informatie**: betekenis geven aan data, is het resultaat van dataverwerking.



3

— Data (2)

Belang van data neemt alsmaar toe.

- Vanaf midden jaren 1990: opkomst online webshops zoals Amazon.
- Vanaf begin jaren 2000: mogelijkheid om inhoud te publiceren op websites.
- Tegenwoordig: heel veel (web)applicaties gekoppeld die data bevatten:
 - Facturatiesoftware, klantenbeheersysteem, studentenadministratie
 - Online webshops en reserveringssystemen
 - Forums en sociale media
 - Internet of Things (IoT):
 - Connectie van toestellen met internet en beheren via bv. smartphone.
 - Gebruik van sensoren: metingen doorsturen naar een databank en deze visualiseren op bv. een website.

4

— Data (3)

Voorbeeld: een webwinkel houdt bij welke klant welke producten bekijkt en/of koopt = ruwe data.

Informatie die we hieruit kunnen halen:

- Hoogste/laagste verkopen in een bepaalde periode van het jaar
→ Prijsbepaling.
- Welke producenten verkopen het meest/minst?
- Heeft de volgorde van productweergave invloed op het aankoopgedrag?
- Koppelen van de initiële zoekterm van de klanten aan het gekochte product om zo betere zoekresultaten te kunnen weergeven
→ Eventueel ook accessoires voorstellen voor het product.
- Klanten indelen in klantenprofielen
→ Gerichte reclame.

5

— Data (4)

Applicaties (geschreven in bv. C#, Python, ...) gebruiken heel wat data, onderscheid tussen:

- **Vluchtige (= volatile) data:**
 - Data wordt geladen in RAM-geheugen (Random-Access Memory).
 - In code: gebruik maken van variabelen.
 - Voordeel: snel.
 - Nadeel: dataverlies bij afsluiten applicatie of toestel.
- **Niet-vluchtige (= non-volatile) data:**
 - Data wordt opgeslagen naar een bestand op de harde schijf.
 - In code: gebruik maken van functionaliteit voor bestandsverwerking.
 - Voordeel: data nog steeds beschikbaar na afsluiten applicatie of toestel.
 - Nadeel: traag bij laden en zoeken van data.



We gaan ons nu (voorlopig) focussen op niet-vluchtige data, dus bestanden.

6

— Data (5)

Praktisch voorbeeld: hoe data van personen met hun gegevens zoals naam, adres, postcode, stad, land, ... gestructureerd opslaan in een bestand?

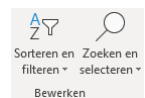
1. Excel-bestand
2. CSV-bestand (Comma Separated Value)
3. XML-bestand (eXtensible Markup Language)
4. JSON-bestand (JavaScript Object Notation)
5. Binair bestand
6. Access-bestand

Doel: straks ook bijhouden welke producten deze personen aangekocht hebben op een webshop.
Zie volgende slides.

— Data (6)

In Excel ook eenvoudige datamanipulaties mogelijk:

- Opzoeken
- Sorteren
- Filteren



1. Excel-bestand:

Naam	Adres	Postcode	Stad	Land
Maria Anders	Obere Strasse 57	12209	Berlijn	Duitsland
Ana Trujillo	Avenida de la Constitución 2222	05021	Mexico-Stad	Mexico
Antonio Moreno	Mataderos 2312	06400	Mexico-Stad	Mexico
Thomas Hardy	120 Hanover Square	E1 6AN	Londen	Verenigd Koninkrijk
Christina Berglund	Berguvsvägen 8	100 05	Stockholm	Zweden

2. CSV-bestand:

Naam;Adres;Postcode;Stad;Land
Maria Anders;Obere Strasse 57;12209;Berlijn;Duitsland
Ana Trujillo;Avenida de la Constitución 2222;05120;Mexico-Stad;Mexico
Antonio Moreno;Mataderos 2312;06400;Mexico-Stad;Mexico
Thomas Hardy;120 Hanover Square;E1 6AN;Londen;Verenigd Koninkrijk
Christina Berglund;Berguvsvägen 8;100 05;Stockholm;Zweden

Beide gevallen: **vlakke datastructuur** (flat file), ok voor eenvoudige data!
Probleem: hoe nu efficiënt aangekochte producten per persoon bijhouden?

— Data (7)

3. XML-bestand:

```
<?xml version="1.0"?>
<personen>
  <persoon>
    <naam>Maria Anders</naam>
    <adres>Obere Strasse 57</adres>
    <postcode>12209</postcode>
    <stad>Berlijn</stad>
    <land>Duitsland</land>
  </persoon>
  <persoon>
    <naam>Ana Trujillo</naam>
    <adres>Avenida de la Constitución 2222</adres>
    <postcode>05021</postcode>
    <stad>Stockholm</stad>
    <land>Zweden</land>
  </persoon>
</personen>
```

Aangekochte producten per persoon:

```
<producten>
  <product>
    <naam>Toetsenbord</naam>
    <prijs>25.49</prijs>
  </product>
  ...
</producten>
```

Door een **geneste (*) datastructuur** te gebruiken is er nu de mogelijkheid om een lijst van aangekochte producten bij te houden.
(*) is impliciet een relationele datastructuur.

9

— Data (8)

4. JSON-bestand:

```
{
  "personen": {
    "persoon": [
      {
        "naam": "Maria Anders",
        "adres": "Obere Strasse 57",
        "postcode": "12209",
        "stad": "Berlijn",
        "land": "Duitsland",
        ...
      },
      {
        "naam": "Ana Trujillo",
        "adres": "Avenida de la Constitución 2222",
        "postcode": "05021",
        "stad": "Mexico-Stad",
        "land": "Mexico"
      },
      ...
    ]
  }
}
```

Aangekochte producten per persoon:

```
"producten": {
  "product": [
    {
      "naam": "Toetsenbord",
      "prijs": 25.49
    },
    ...
  ]
}
```

Idem als voorgaand geval.

10

— Data (9)

5. **Binair bestand:** bevat bv. geserialiseerde objecten.
 - Relationale datastructuur.
6. **Access-bestand:** bevat meerdere tabellen.
 - Relationale datastructuur.

Problemen:

- Bestandtype 1 en 2: vlakke datastructuur
- Alle bestandtypes: bestand kan maar door **één proces/applicatie** tegelijkertijd **geschreven** worden.
- Bestandtype 1 t.e.m. 5:
 - Traag zoekproces om data te vinden.
 - Traag bij toevoegen en wijzigen data.

Oplossing: gebruik maken van een databanksysteem.

11

— Data (10)

Type	Geneste/relatieve datastructuur?	Multi gebruikers?	Efficient en snel opslaan en ophalen data?	Crash recovery?	Gebruik
Excel	X	X	X	X	Kleine hoeveelheid (voornamelijk numerieke) data die verwerkbaar zijn met Excel-basisfuncties
CSV	X	X	X	X	
XML	V	X	X	X	Grotere hoeveelheid numerieke en alfanumerieke data die verwerkbaar zijn via een programmeertaal
JSON	V	X	X	X	
Binair	V	X	X	X	
Access	V	X	V	X	Grotere hoeveelheid numerieke en alfanumerieke data die verwerkbaar zijn d.m.v. queries in een programmeertaal
→ Databanksysteem	V	V	V	V	

12

— Databank

Doel van een **databank** (database):

- **Digitaal** opgeslagen **archief**
- Kan flexibel **geraadpleegd** en **gebruikt** worden door meerdere gebruikers tegelijkertijd:
 - Doorzoeken van gegevens
 - Wijzigen van gegevens
 - Toevoegen van gegevens
 - Verwijderen van gegevens
- Speelt dus een belangrijke rol voor het **archiveren** en het **actueel houden** van data



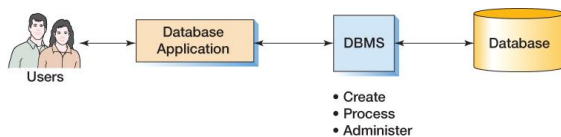
13

hogeschool
vives

— Databankstelsysteem ⁽¹⁾

Een **databankstelsysteem** bestaat uit 4 componenten:

1. **Gebruikers:** gewone gebruikers en administrators.
2. **Databankapplicatie:**
 - Gewone gebruikers: kant-en-klare visuele datasoftware (bv. voor rapportage) of een (zelfgemaakte) computerprogramma geschreven in bv. Python, C#, ...
 - Administrators: gebruiken programma om DBMS te visualiseren, bv. MySQL Workbench, SQL Server Management Studio, ...



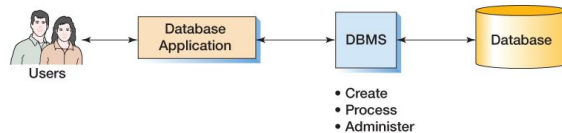
14

hogeschool
vives

— Databanksysteem (1)

Een **databanksysteem** bestaat uit 4 componenten:

3. **DBMS** (DataBase Management System): applicatie (op de achtergrond) om databanken te creëren, te gebruiken en te beheren.
 - Belangrijke taak is het handhaven van de integriteit van de gegevens:
 - Voldoen aan regels
 - Consistentie van de gegevens
4. **Databank** zelf:
 - Bevat bij elkaar horende data/gegevens, opgeslagen in een set van onderling gerelateerde tabellen.

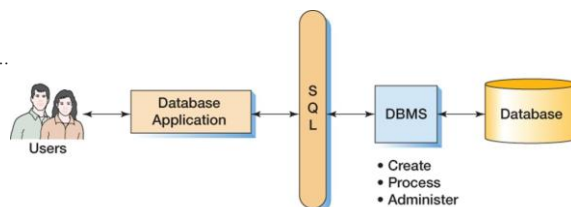


15

— Databanksysteem (2)

Er zijn 2 types van databanken:

- **SQL-databanken**: relationele databanken → wordt behandeld in deze cursus.
 - Voorbeelden DBMS'en: Oracle, Microsoft SQL Server, MySQL, ...
 - Soms spreekt men hier ook van de term **RDBMS** (Relational DataBase Management System).
 - De databank is accessible via SQL (Structured Query Language), een internationaal erkende standaard databanktaal die door alle commerciële DBMS'en wordt gebruikt.
 - Let wel: kleine verschillen van de SQL-syntaxis zijn mogelijk.
- **NoSQL-databanken**
 - Voorbeelden: MongoDB, InfluxDB, ...



16

— RDBMS (1)

Taken:

- Aanmaken van databanken.
- Aanmaken van tabellen.
- Aanmaken van extra structuren zoals indices.
- Bevat mogelijkheden voor CRUD (create, read, update en delete) datamanipulaties.
- Onderhouden van databankstructuren.
- Regels instellen (constraints).
- Regelt de toegang voor multigebruikers.
- Laat beveiliging toe (permissieniveaus).
- Laat toe om databank te back-uppen en te herstellen.

17

— RDBMS (2)

Inhoud:

- Tabellen met data
- Metadata: info bijhouden over de data
- Indices
- Stored procedures
- Triggers
- Views
- Beveiliging van data
- Backup/herstel data

18